

*Introduction to Databases 2024-2025 Project:
E-commerce Database Integration*

PROJECT REPORT

Project Evaluation Date:

15/01/2025

Prepared by:

Metin Bingöl – 05220000312

Altuğ Emre Tosun – 05220000295

Group

53

Analysis

Data requirements for each website:

Data requirements for Amazon

The diagram has **USER** superclass, each uniquely identified by a User id. Users have the following attributes: Fname (First name), Lname (Last name), Email, Password, Phone number, Birth date, Join date and Sex. And there are two subclasses (Overlapping) of **USER**: **CUSTOMER** and **SELLER**. These subclasses inherit the primary key and other attributes from the **USER** superclass. Customer has only primary key User id, each seller has unique User id, Contact number and Rating. Customers can initiate the purchasing process (**USES**, **N:1**) and link a credit card to their account for payments. Additionally, they maintain a list of items (**LISTS**) (**1:N**) to track preferred products. Each list must have at least 1 customer. Sellers offer products (**SELLS**), where one seller can manage multiple products (**1:N**). And sellers may have **FBM**.

Products (**PRODUCT**) are tracked with the following attributes: Unique Product id, Product name, Description, Brand and Product type (Digital or Physical product). Products are categorized under **CATEGORY** (**N:1**), which includes: Category id, Cname (Category Name) and Cdescription (Category Description). Subcategories reference higher-level categories (Sub category id). One product belongs to only one category. But each category can have any number of products.

Products are sold by various sellers with specific prices under the **SHOP_PRODUCT**. Attributes include: Unique SKU (Stock keeping Unit), Price, Discount, Stock and Added date. Also **SHOP_PRODUCT** is a weak entity and its strong entites are: **PRODUCT** and **SELLER**. Customers interact with a shopping cart (**CART**), identified by a CartID. Each cart can hold multiple items (**1:N**) (**CART_ITEM**), which depends on **CART** and **SHOP_PRODUCT**. Attributes include: Cart total amount, Total items and Updated date. Customers owns their cart (**OWNED_BY**) (**1:1**) and finalize orders from their cart. **CART_ITEM** is a weak entiy and its strong entites are: **CART** and **SHOP_PRODUCT**. And the attributes are: Cartitem id, Price and Quantity. Customers can provide feedback on products by leaving reviews (**REVIEWS**) (**M:N**) with attributes: Rating, Comment and Review date. Each customer writes any number of reviews and also each shop product can have any number of reviews. Orders (**ORDER**) are managed and identified by a unique OrderID. Attributes include: Order date, Order status (Reflects the current state of the orde), Total amount, Delivery type, Premium benefit (Boolean). Each order is associated with a customer who places it. Orders are (**TRACKED_BY**) (**N:1**) by logistics carriers. And cart is converted to order and the order paid by (**PAID_BY**) (**1:1**) payment method. Each order must be paid by some payment method. The system oversees logistics operations through the **LOGISTICS** entity, identified by a unique LogisticID. Attributes include: Tracking number, Estimated delivery time, Status (Tracks the current

state of delivery), Shipping fee, Priority shipping (Boolean), Shipping Company. Payments (**PAYMENT_METHOD**) are processed and identified by a unique Payment method id. Attributes include: Method name (Credit card, Bank card, Gift card), Provider name (Mastercard, Visa, ...), Payment info (Can include card number etc.). The **ADDRESS** entity holds the attributes related to location information: Unique Address id, Country, City, District, Street and Postal code. The logistics operations deliver the orders to the relevant address (**DELIVERED_TO**) (M:N). Each customer (**HAS_ADDR**) has at least 0 address in the system and each address has at least 1 customer. This relationship (**HAS_ADDR**) includes the Address type attribute for billing address and shipping address. The **WAREHOUSE** superclass has unique Warehouse id, Location and fee. And subclasses of warehouse are (disjoint) **FBA** and **FBM**. Each FBM must associate with one seller. And each seller may have any number of FBM. **FBA (Fulfilled by Amazon)**: Warehouses managed and operated by Amazon. **FBM (Fulfilled by Merchant)**: Warehouses managed by individual merchants. Logistics services operate in multiple Warehouses, indicating that logistics providers facilitate operations at specific warehouses. This relationship (**OPERATES_IN**) is **many-to-many (N:M)**, meaning multiple warehouses can be serviced by multiple logistics providers. Each Warehouse may manage multiple Orders, ensuring that products for those orders are stored and shipped from that warehouse. This relationship (**HANDLED_BY**) is **one-to-many (1:N)**, meaning a single warehouse can be responsible for fulfilling multiple orders. There are two types of customers on Amazon: Prime customers and regular customers. We represented this distinction by dividing the **CUSTOMER** entity into two subclasses: **PRIME_CUSTOMER** and **REGULAR_CUSTOMER**. Prime customers have some advantages over regular customers, which we captured with a Benefit attribute. Additionally, we included Start Date and End Date attributes to represent the subscription period for Prime customers. In the provided diagram, the **USER** and **MESSAGE** entities are connected through two relationships: **SENDS AND RECEIVED_BY**. **MESSAGE** has the following attributes: MessageID, Content (The actual text or content of the message) and timestamp. **The two relationship is many-to-many (M:N)**, meaning multiple messages can be sends/received by multiple users.

Data requirements for Trendyol

The system tracks customers (CUSTOMER), each identified by a unique customer ID (UserID). Attributes include: Name (Name), consisting of: First Name (FirstName), Last Name (LastName), Email (Email), Date of Birth (Birthday), Gender (Sex), Physical characteristics: Weight (Weight), Height (Height), Address (Address). Customers can follow other customers (FOLLOWING CONNECTIONS), which allows for social engagement between users. Customers starts the process of order as (Sends the order) and also can add Credit card in case they want to make a payment. Customers follow sellers (FOLLOWS) to stay updated on products. Customers maintain favorite items (FAVORITE ITEMS), with a collection of preferred products.

The system stores information about sellers (SELLER), each identified by a unique SellerID. Attributes include: Name (Name), Contact information (ContactInfo), Location (Location), Number of subscribers (Subscribers), Rating (Rating). Sellers sell products (SELLS), where one seller can sell multiple products.

The system tracks products (PRODUCT) with the following attributes: Product ID (ProductID), which uniquely identifies each product. Product Name (PName), Description (Description), Brand (Brand), Stock availability (Stock), Popularity status (Popular), Classification: Domestic or imported (Domestic), Product images (ImageURLs).

Products are organized into categories (CATEGORY), which may include subcategories: Each category has a unique CategoryID and a CategoryName. Subcategories reference a higher-level category (UpperCategoryID). Products are also listed with prices by different sellers under the Shop Product (SHOP PRODUCT) relationship: Price (Price). Campaign details, if applicable (CAMPAIGN).

Customers interact with a shopping cart (CART), identified by a CartID. Each cart can hold multiple cart items (CART ITEM), which is a weak entity dependent on CART and PRODUCT. Attributes include: Quantity of the product in the cart (Quantity). Customers add products to their cart (ADDS TO). CART ITEM links a cart to specific products.

Customers can leave reviews (REVIEWS) for products, with attributes: Rating (Rating), Comments (Comment), Review date (Date). Customers can also apply coupons (USE COUPON) during purchases, with attributes: Discount amount (DiscountAmount).

Orders are managed as orders (ORDER), with each order identified by a unique OrderID. Attributes include: Delivery date (DeliveryDate). Status (Status). Shipping fee (ShippingFee). Orders are associated with customers who place them. Orders are delivered by logistics carriers (DELIVERY), as described below.

The system manages logistics operations through the logistics (LOGISTICS) entity, identified by a unique LogisticID. Attributes include: Tracking number (TrackingNumber). Estimated delivery time (EstimatedTime), Status (Status), Geolocation (GeoLocation). Logistics services are provided by carriers (CARRIER), where carriers are entities with: CarrierID (CarrierID), Name (Name), Service type (ServiceType), Rating (Rating). Carriers carry orders (CARRY), ensuring product delivery.

Payments are recorded in the payment (PAYMENT) entity, identified by a unique PaymentID. Attributes include: Payment date (Date), Payment amount (Amount), Installment details (Installment). Payments can be made through different methods: Credit cards (CREDIT CARD), with attributes: CardID (CardID), Card type (CardType), Shopping credits (SHOPPING CREDIT), with attributes: Bank Name (BankName).

Data requirements for Hepsiburada

The system tracks customers (CUSTOMER), each identified by a unique customer ID (CustomerID). The attributes stored for each customer include: Name (Name), which is composed of first name (FirstName) and lastname(LastName), Email (Email), Gender (Gender), Date of birth (DoB), Addresses (Address), which may include multiple addresses for billing and shipping purposes. Membership in a premium subscription program (HEPSIBURADA PREMIUM), which includes: Benefits (Benefits). Subscription end date (EndDate). Customers can follow multiple sellers (FOLLOWS) to receive updates on their products. Customers maintain a list of favorite products (FAVORITE ITEMS). Customers can use discount coupons (USE COUPON), which have a discount amount. Customers can create shopping lists (LISTS), where each list contains multiple products. The LIST entity includes: List name (ListName), Date of creation (CreationDate).

The system stores information about sellers (SELLER), each identified by a unique seller ID (SellerID). Attributes for each seller include: Name (Name), Contact information (ContactInfo), Location (Location), Rating (Rating), Number of subscribers (Subscribers). Sellers offer multiple products (SELLS).

The products (PRODUCT) tracked in the system include the following attributes: A unique product ID (ProductID), Product name (PName), Description (Description), Brand (Brand), Popularity status (Popular), Stock levels (Stock), An indicator of whether the product is domestic (Domestic), Associated product images (ImageURLs), Products belong to categories (CATEGORY) and subcategories (SUB CATEGORY). Each category (CATEGORY) is identified by: A unique ID (CategoryID), A name (CategoryName), An optional reference to a higher-level category (UpperCategoryID). Products can belong to multiple categories (IS IN). Products can have prices from different sellers (SHOP PRODUCT). The SHOP PRODUCT relationship tracks: Price (Price) offered by each seller for a specific product.

Customers can add products to their shopping cart (CART). Each cart is uniquely identified by a cart ID (CartID). A cart can contain multiple cart items (CART ITEM), which

is a weak entity related to CART and PRODUCT. CART ITEM is uniquely identified by: The cart ID (CartID), The product ID (ProductID). Attributes of CART ITEM include: Quantity (Quantity). Customers can add products to their cart (ADDS TO). CART ITEM exists only in the context of a specific CART and PRODUCT. Orders (ORDER) are created when customers proceed to purchase items in their cart. Each order is uniquely identified by an order ID (OrderID). Attributes of orders include: Delivery date (DeliveryDate), Shipping fee (ShippingFee), Current order status (Status). Each order is linked to a payment (CONCLUDES), Each order is delivered through logistics (DELIVER).

Payments for orders are tracked in the payment (PAYMENT) entity. Each payment has a unique ID (PaymentID). Attributes include: Payment amount (Amount). Payment date (Date). Number of installments (Installment). Payments can be made through multiple methods (PAID WITH), including credit cards (CREDIT CARD). CREDIT CARD is uniquely identified by: Card ID (CardID). Attributes of CREDIT CARD include: Card type (CardType), Bank name (BankName).

The system tracks logistics (LOGISTICS) to facilitate order deliveries. Each logistics record has a unique logistics ID (LogisticID). Attributes include: Tracking number (TrackingNumber), Estimated delivery time (EstimatedTime), Current status (Status). Orders are delivered by a specific logistics provider (CARRIES). Deliveries are handled by a carrier (CARRIER), which has attributes: Carrier ID (CarrierID), Carrier name (Name), Type of service provided (ServiceType), Carrier rating (Rating).

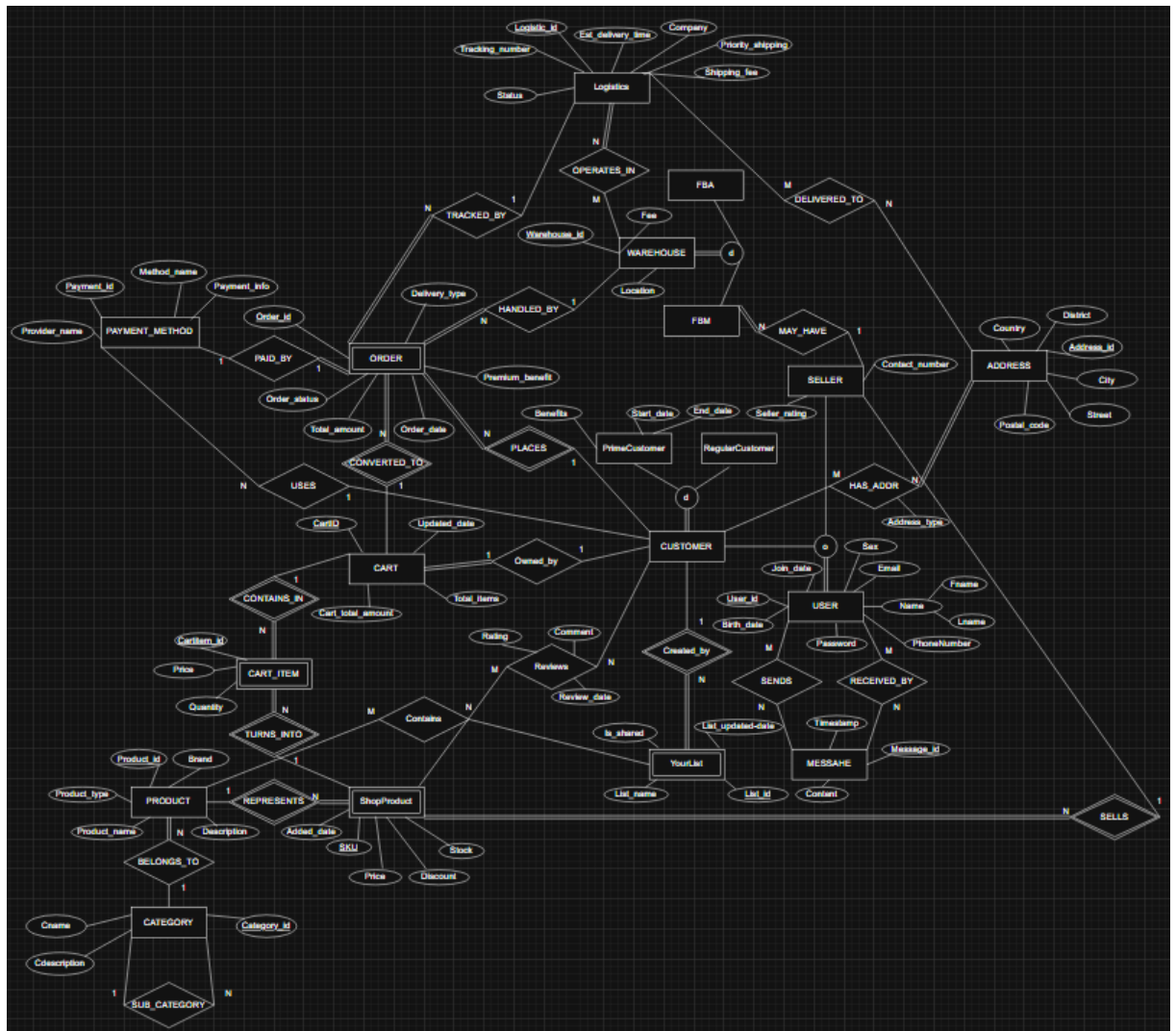
Customers can leave reviews (REVIEWS) for products they have purchased. Each review includes: A rating (Rating), A comment (Comment), The date of the review (Date). Customers can ask questions (QUESTION) about products, and sellers or other customers can provide responses (RESPONSE). The system supports payments through a dedicated wallet service called Hepsipay (HEPSIPAY). Hepsipay stores: A unique wallet ID (HepsilID), The balance (HepsiparaBalance).

The system manages the delivery of orders through carriers. For each logistics process, the following details are tracked: A unique logistics ID. Tracking number for the order shipment. Estimated delivery time. Current status of the delivery (e.g., in transit, delivered). Each order is delivered by a carrier, with details of the carrier including: Carrier ID, Carrier name, Type of service provided, Carrier rating based on performance.

DESIGN-CONCEPTUAL DESIGN

1) EER diagrams for each website:

Amazon:



The diagram is a complex ER model for an e-commerce system, divided into three main sections: CARRIER, LOGISTICS, and PROVIDER. It includes various entities, attributes, and relationships.

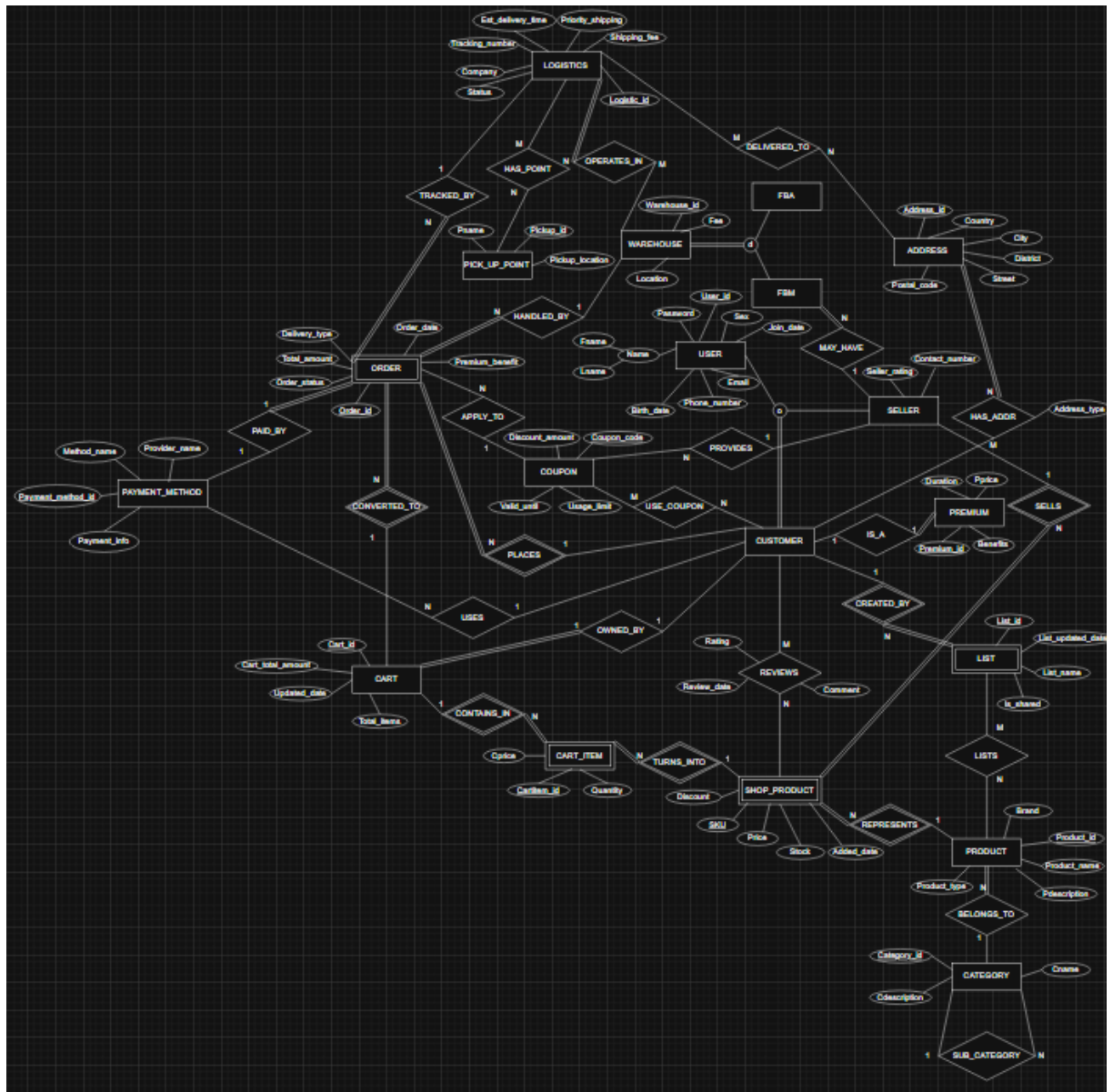
Entities and Attributes:

- CARRIER:** Name, Rating, ServiceArea.
- LOGISTICS:** Transportation, TransportationID, LocationID.
- PROVIDER:** (Abstract entity)
- BELLA:** (Abstract entity)
- SELLER:** Name, Location, ContactInfo, Education, Rating.
- CATEGORY:** CategoryName, SpecialCategory.
- PRODUCT:** Name, ProductID, Price, Description, Stock, Domestic.
- CUSTOMER:** Name, Address, Email, Phone, CreditCard, OrderID, InventoryID, InventoryDate, InventorySize, InventoryWeight, InventoryVolume, InventoryValue, InventoryColor, InventoryShape, InventorySize, InventoryWeight, InventoryVolume, InventoryValue, InventoryColor, InventoryShape.
- ORDER:** OrderID, Status, Description, CarrierID.
- PAYMENT:** PaymentID, Amount, Date, InvoiceID.
- CREDIT CARD:** CreditCardID, CardType, CardName.
- SHOP PRODUCT:** Price.

Relationships and Cardinalities:

- CARRIER** (1) **LOGISTICS** (N)
- LOGISTICS** (1) **PROVIDER** (N)
- PROVIDER** (1) **BELLA** (N)
- BELLA** (1) **SELLER** (N)
- SELLER** (1) **CATEGORY** (N)
- CATEGORY** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD** (N)
- CREDIT CARD** (1) **ORDER** (N)
- ORDER** (1) **SHOP PRODUCT** (N)
- SHOP PRODUCT** (1) **PRODUCT** (N)
- PRODUCT** (1) **CUSTOMER** (N)
- CUSTOMER** (1) **ORDER** (N)
- ORDER** (1) **PAYMENT** (N)
- PAYMENT** (1) **CREDIT CARD**

2) Combination of all EER diagrams for each website:



Firstly, we added the main features that every e-commerce site should have. Afterwards, we added some specific features that we consider important for e-commerce sites. For Amazon, we added FBA and FBM. For Trendyol, we added Pick up points. For Hepsiburada we added premium customer account.

DESIGN-LOGICAL MODEL

Convert EER diagram into a relational model:

Methodology:

1st Iteration

Step 1

USER(User_id, Fname, Lname, Email, Password, Phone_number, Birth_date, Join_date, Sex)

WAREHOUSE(Warehouse_id, Location, Fee)

LOGISTICS(Logistic_id, Tracking_number, Company, Status, Est_delivery_time, Shipping_fee, Priority_shipping)

ADDRESS(Address_id, Country, City, District, Street, Postal_code)

PAYMENT_METHOD(Payment_method_id, Method_name, Provider_name, Payment_info)

CART(Cart_id, Total_items, Cart_total_amount, Updated_date)

PRODUCT(Product_id, Product_name, Brand, Description, Product_type)

CATEGORY(Category_id, Cname, Cdescription)

COUPON(Coupon_code, Discount_amount, Usage_limit, Valid_until)

PICK_UP_POINT(Pickup_id, Pname, Pickup_location)

PREMIUM(Premium_id, Benefits, Pprice, Duration)

Step 2 -

Step 3 -

Step 4 -

PRODUCT(Product_id, CATEGORY.Category_id, Product_name, Brand, Description, Product_type)

CATEGORY(Category_id, Cname, Cdescription, Sub_category_id)

Step 5

DELIVERED_TO(LOGISTICS.Logistic_id, ADDRESS.Address_id)

HAS_POINT(LOGISTICS.Logistic_id, PICK_UP_POINT.Pickup_id)

OPERATES_IN(LOGISTICS.Logistic_id, WAREHOUSE.Warehouse_id)

Step 6 -

Step 7 -

Step 8 -

CUSTOMER(User_id) SELLER(User_id, Seller_rating, Contact_number)
FBA(Warehouse_id) FBM(Warehouse_id)

Step 9 -

2nd Iteration

Step 1 -

Step 2 -

ORDER(Order_id, CUSTOMER.User_id, CART.Cart_id, Order_status, Order_date,
Delivery_type, Total_amount, Premium_benefit)
SHOP_PRODUCT(SKU, PRODUCT.Product_id, SELLER.User_id, Price, Stock, Discount,
Added_date)
LIST(List_id, CUSTOMER.User_id, List_name, List_updated_date, Is_shared)

Step 3

ORDER(Order_id, CUSTOMER.User_id, CART.Cart_id,
PAYMENT_METHOD.Payment_method_id, Order_status, Order_date, Delivery_type,
Total_amount, Premium_benefit)
CART(Cart_id, CUSTOMER.User_id, Total_items, Cart_total_amount, Updated_date)
PREMIUM(Premium_id, CUSTOMER.User_id, Benefits, Pprice, Duration)

Step 4

ORDER(Order_id, CUSTOMER.User_id, CART.Cart_id,
PAYMENT_METHOD.Payment_method_id, WAREHOUSE.Warehouse_id,
LOGISTICS.Logistic_id, COUPON.Coupon_id, Order_status, Order_date, Delivery_type,
Total_amount, Premium_benefit)
PAYMENT_METHOD(Payment_method_id, CUSTOMER.User_id, Method_name,
Provider_name, Payment_info)
FBM(Warehouse_id, SELLER.User_id)
COUPON(Coupon_code, SELLER.User_id, Discount_amount, Usage_limit, Valid_until)

Step 5

HAS_ADDR(ADDRESS.Address_id, CUSTOMER.User_id, Address_type)
USE_COUPON(COUPON.Coupon_id, SELLER.User_id)

Step 6 -

Step 7 -

Step 8 -

Step 9 -

3rd Iteration

Step 1 -

Step 2

CART_ITEM(Cartitem_id, CART.Cart_id, SKU, PRODUCT.Product_id, SELLER.User_id,
Cprice, Quantity)

Step 3 -

Step 4 -

Step 5 -

LISTS(List_id, CUSTOMER.User_id, PRODUCT.Product_id)

REVIEWS(CUSTOMER.User_id, SKU, PRODUCT.Product_id, SELLER.User_id, Rating,
Comment, Review_date)

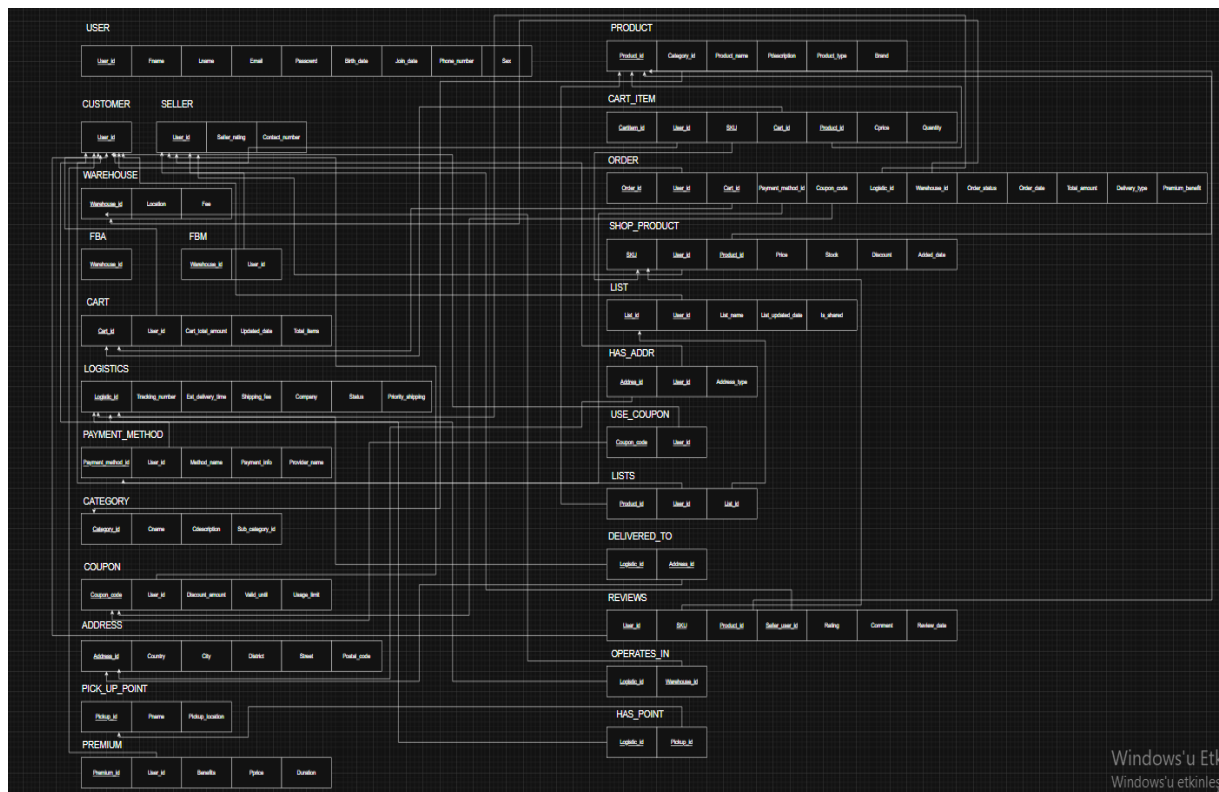
Step 6 -

Step 7 -

Step 8 -

Step 9 -

Relational Database Tables:



IMPLEMENTATION-PHYSICAL MODEL

1) SQL scripts (DDL statements) for creating the database and its relational model:

```
CREATE TABLE USER (  
    User_id INT PRIMARY KEY AUTO_INCREMENT,  
    Fname VARCHAR(50),  
    Lname VARCHAR(50),  
    Email VARCHAR(100) UNIQUE,  
    Password VARCHAR(255),  
    Birth_date DATE,  
    Join_date DATE,  
    Phone_number VARCHAR(15),  
    Sex ENUM('M', 'F', 'Other')  
);
```

```
CREATE TABLE CUSTOMER (  
    User_id INT PRIMARY KEY,  
    FOREIGN KEY (User_id) REFERENCES USER(User_id)  
);
```

```
CREATE TABLE SELLER (  
    User_id INT PRIMARY KEY,  
    Seller_rating DECIMAL(3, 2),  
    Contact_number VARCHAR(15),  
    FOREIGN KEY (User_id) REFERENCES USER(User_id)  
);
```

```
CREATE TABLE WAREHOUSE (  
    Warehouse_id INT PRIMARY KEY AUTO_INCREMENT,  
    Location VARCHAR(100),  
    Fee DECIMAL(10, 2)  
);
```

```
CREATE TABLE FBA (  
    Warehouse_id INT PRIMARY KEY,
```

```
FOREIGN KEY (Warehouse_id) REFERENCES WAREHOUSE(Warehouse_id)

);
```

```
CREATE TABLE FBM (

    Warehouse_id INT PRIMARY KEY,

    User_id INT,

    FOREIGN KEY (Warehouse_id) REFERENCES WAREHOUSE(Warehouse_id),

    FOREIGN KEY (User_id) REFERENCES SELLER(User_id)

);
```

```
CREATE TABLE CART (

    Cart_id INT PRIMARY KEY AUTO_INCREMENT,

    User_id INT,

    Cart_total_amount DECIMAL(10, 2),

    Updated_date DATETIME,

    Total_items INT,

    FOREIGN KEY (User_id) REFERENCES CUSTOMER(User_id)

);
```

```
CREATE TABLE LOGISTICS (

    Logistic_id INT PRIMARY KEY AUTO_INCREMENT,

    Tracking_number VARCHAR(50),

    Est_delivery_time DATETIME,

    Shipping_fee DECIMAL(10, 2),

    Company VARCHAR(50),

    Status VARCHAR(20),

    Priority_shipping BOOLEAN

);
```

```
CREATE TABLE PAYMENT_METHOD (

    Payment_method_id INT PRIMARY KEY AUTO_INCREMENT,

    User_id INT,

    Method_name VARCHAR(50),

    Payment_info TEXT,

    Provider_name VARCHAR(50),

    FOREIGN KEY (User_id) REFERENCES CUSTOMER(User_id)

);
```

```
CREATE TABLE CATEGORY (  
    Category_id INT PRIMARY KEY AUTO_INCREMENT,  
    Category_name VARCHAR(50),  
    Category_description TEXT,  
    Sub_category_id INT  
);
```

```
CREATE TABLE COUPON (  
    Coupon_code VARCHAR(20) PRIMARY KEY,  
    User_id INT,  
    Discount_amount DECIMAL(10, 2),  
    Valid_until DATE,  
    Usage_limit INT,  
    FOREIGN KEY (User_id) REFERENCES SELLER(User_id)  
);
```

```
CREATE TABLE ADDRESS (  
    Address_id INT PRIMARY KEY AUTO_INCREMENT,  
    Country VARCHAR(50),  
    City VARCHAR(50),  
    District VARCHAR(50),  
    Street VARCHAR(100),  
    Postal_code VARCHAR(10)  
);
```

```
CREATE TABLE PICK_UP_POINT (  
    Pickup_id INT PRIMARY KEY,  
    PName VARCHAR(50),  
    Pickup_location VARCHAR(100)  
);
```

```
CREATE TABLE PREMIUM (  
    Premium_id INT PRIMARY KEY AUTO_INCREMENT,  
    User_id INT,  
    Benefits TEXT,  
    Price DECIMAL(10, 2),  
    Duration INT,  
    FOREIGN KEY (User_id) REFERENCES CUSTOMER(User_id)
```


);

CREATE TABLE PRODUCT (

Product_id INT PRIMARY KEY AUTO_INCREMENT,

Category_id INT,

Product_name VARCHAR(100),

Product_description TEXT,

Product_type VARCHAR(50),

Brand VARCHAR(50),

FOREIGN KEY (Category_id) REFERENCES CATEGORY(Category_id)

);

CREATE TABLE SHOP_PRODUCT (

SKU INT,

User_id INT,

Product_id INT,

Price DECIMAL(10, 2),

Stock INT,

Discount DECIMAL(10, 2),

Added_date DATETIME,

PRIMARY KEY (SKU, User_id, Product_id),

FOREIGN KEY (User_id) REFERENCES SELLER(User_id),

FOREIGN KEY (Product_id) REFERENCES PRODUCT(Product_id),

CHECK(STOCK > -1),

CHECK(Price > 0)

);

CREATE TABLE CART_ITEM (

CartItem_id INT,

User_id INT,

Cart_id INT,

SKU INT,

Product_id INT,

Cprice DECIMAL (10, 2),

Quantity INT,

PRIMARY KEY (CartItem_id, User_id, SKU, Cart_id, Product_id),

FOREIGN KEY (SKU) REFERENCES SHOP_PRODUCT(SKU),

FOREIGN KEY (User_id) REFERENCES SELLER(User_id),

```
FOREIGN KEY (Cart_id) REFERENCES CART(Cart_id),  
FOREIGN KEY (Product_id) REFERENCES PRODUCT(Product_id),  
CHECK(Quantity > 0)  
);
```

```
CREATE TABLE `ORDER` (  
    Order_id INT,  
    User_id INT,  
    Cart_id INT,  
    Payment_method_id INT,  
    Coupon_code VARCHAR(20),  
    Logistic_id INT,  
    Warehouse_id INT,  
    Order_status VARCHAR(50),  
    Order_date DATETIME,  
    Total_amount DECIMAL(10, 2),  
    Delivery_type VARCHAR(50),  
    Premium_benefit BOOLEAN,  
    PRIMARY KEY (Order_id, User_id, Cart_id),  
    FOREIGN KEY (User_id) REFERENCES CUSTOMER(User_id),  
    FOREIGN KEY (Cart_id) REFERENCES CART(Cart_id),  
    FOREIGN KEY (Payment_method_id) REFERENCES PAYMENT_METHOD(Payment_method_id),  
    FOREIGN KEY (Coupon_code) REFERENCES COUPON(Coupon_code),  
    FOREIGN KEY (Logistic_id) REFERENCES LOGISTICS(Logistic_id),  
    FOREIGN KEY (Warehouse_id) REFERENCES WAREHOUSE(Warehouse_id)  
);
```

```
CREATE TABLE LIST (  
    List_id INT,  
    User_id INT,  
    List_name VARCHAR(100),  
    List_updated_date DATETIME,  
    Is_shared BOOLEAN,  
    PRIMARY KEY (List_id, User_id),  
    FOREIGN KEY (User_id) REFERENCES CUSTOMER(User_id)  
);
```

```
CREATE TABLE HAS_ADDR (  
    -- Table structure to be defined here --  
);
```

```
Address_id INT,  
  
User_id INT,  
  
Address_type ENUM('Billing Address','Shipping Address'),  
  
PRIMARY KEY (Address_id, User_id),  
  
FOREIGN KEY (Address_id) REFERENCES ADDRESS(Address_id),  
  
FOREIGN KEY (User_id) REFERENCES CUSTOMER(User_id)  
);
```

```
CREATE TABLE USE_COUPON (  
  
    Coupon_code VARCHAR(20),  
  
    User_id INT,  
  
    PRIMARY KEY (Coupon_code, User_id),  
  
    FOREIGN KEY (Coupon_code) REFERENCES COUPON(Coupon_code),  
  
    FOREIGN KEY (User_id) REFERENCES CUSTOMER(User_id)  
);
```

```
CREATE TABLE LISTS (  
  
    Product_id INT,  
  
    User_id INT,  
  
    List_id INT,  
  
    PRIMARY KEY (Product_id, User_id, List_id),  
  
    FOREIGN KEY (Product_id) REFERENCES PRODUCT(Product_id),  
  
    FOREIGN KEY (User_id) REFERENCES CUSTOMER(User_id),  
  
    FOREIGN KEY (List_id) REFERENCES LIST(List_id)  
);
```

```
CREATE TABLE DELIVERED_TO (  
  
    Logistic_id INT,  
  
    Address_id INT,  
  
    PRIMARY KEY (Logistic_id, Address_id),  
  
    FOREIGN KEY (Logistic_id) REFERENCES LOGISTICS(Logistic_id),  
  
    FOREIGN KEY (Address_id) REFERENCES ADDRESS(Address_id)  
);
```

```
CREATE TABLE REVIEWS (  
  
    Customer_user_id INT,  
  
    SKU INT,  
  
    Product_id INT,
```

```

    Seller_user_id INT,

    Rating DECIMAL(3, 2),

    Comment TEXT,

    Review_date DATETIME,

    PRIMARY KEY (Seller_user_id, Customer_user_id, SKU, Product_id),

    FOREIGN KEY (Customer_user_id) REFERENCES CUSTOMER(User_id),

    FOREIGN KEY (SKU) REFERENCES SHOP_PRODUCT(SKU),

    FOREIGN KEY (Product_id) REFERENCES PRODUCT(Product_id),

    FOREIGN KEY (Seller_user_id) REFERENCES SELLER(User_id),

    CHECK (Rating >= 0 AND Rating <= 5)

);

```

```

CREATE TABLE OPERATES_IN (

    Logistic_id INT,

    Warehouse_id INT,

    PRIMARY KEY (Logistic_id, Warehouse_id),

    FOREIGN KEY (Logistic_id) REFERENCES LOGISTICS(Logistic_id),

    FOREIGN KEY (Warehouse_id) REFERENCES WAREHOUSE(Warehouse_id)

);

```

```

CREATE TABLE HAS_POINT (

    Logistic_id INT,

    Pickup_id INT,

    PRIMARY KEY (Logistic_id, Pickup_id),

    FOREIGN KEY (Logistic_id) REFERENCES LOGISTICS(Logistic_id),

    FOREIGN KEY (Pickup_id) REFERENCES PICK_UP_POINT(Pickup_id)

);

```

2) Populate the database:

```

INSERT INTO USER (Fname, Lname, Email, Password, Birth_date, Join_date, Phone_number, Sex) VALUES

('Murat Osman', 'Ünalır', 'murat.osman.unalir@ege.edu.tr', 'password123', '1971-05-15', '2020-02-03', '555-1234', 'M'),

('Cem', 'Alici', 'cem.alici@ege.edu.tr', 'password456', '1998-08-20', '2024-06-07', '555-5678', 'M'),

('Sevtap Duman', 'Kaymaz', 'sevtap.duman@ege.edu.tr', 'password789', '1985-12-25', '2022-03-10', '555-9012', 'F'),

('Emine', 'Sezer', 'emine.sezer@ege.edu.tr', 'hashed_password', '1979-01-01', '2021-01-14', '553-1238', 'F');

```

INSERT INTO CUSTOMER (User_id) VALUES

(2),

(3);

INSERT INTO SELLER (User_id, Seller_rating, Contact_number)

VALUES

(1, 0, '1234567890'),

(4, 0, '2345978901');

INSERT INTO WAREHOUSE (Location, Fee)

VALUES

('İzmir', 35.00),

('İstanbul', 45.00),

('Ankara', 30.00);

INSERT INTO FBA (Warehouse_id)

VALUES

(1);

INSERT INTO FBM (Warehouse_id, User_id)

VALUES

(2, 1),

(3, 4);

INSERT INTO CART (User_id, Cart_total_amount, Updated_date, Total_items)

VALUES

(2, 0.00, '2023-03-15 10:30:00', 0),

(3, 0.00, '2023-03-16 14:00:00', 0);

INSERT INTO LOGISTICS (Tracking_number, Est_delivery_time, Shipping_fee, Company, Status, Priority_shipping) VALUES

('TRACK123', '2023-04-01 12:00:00', 10.50, 'MNG Kargo', 'In Transit', TRUE),

('TRACK456', '2023-04-02 15:00:00', 5.00, 'Yurtiçi', 'Delivered', FALSE);

INSERT INTO PAYMENT_METHOD (User_id, Method_name, Payment_info, Provider_name) VALUES

(2, 'Credit Card', '1234-5678-9012-3456', 'Visa'),

(3, 'Gift Card', '15\$', 'Amazon');

```
INSERT INTO CATEGORY (Category_name, Category_description, Sub_category_id) VALUES
('Electronics', 'Devices and gadgets', NULL),
('Books', 'Printed and digital books', NULL);
```

```
INSERT INTO COUPON (Coupon_code, User_id, Discount_amount, Valid_until, Usage_limit) VALUES
('DISCOUNT10', 1, 10.00, '2025-09-08', 5),
('DISCOUNT5', 4, 5.00, '2025-06-01', 3);
```

```
INSERT INTO ADDRESS (Country, City, District, Street, Postal_code) VALUES
('Türkiye', 'İzmir', 'Bornova', '123 Sokak', '35080'),
('Türkiye', 'İzmir', 'Konak', '456 Market Sokak', '35250'),
('Türkiye', 'İzmir', 'Balçova', '290 Sokak', '35330');
```

```
INSERT INTO PICK_UP_POINT (Pickup_id, PName, Pickup_location) VALUES
(1,'Pickup Point A', '123 Main St, City A'),
(2,'Pickup Point B', '456 Second St, City B'),
(3,'Pickup Point C', '789 Third St, City C');
```

```
INSERT INTO PREMIUM (User_id, Benefits, Price, Duration) VALUES
(2, 'Free shipping, exclusive deals', 9.99, 12);
```

```
INSERT INTO PRODUCT (Category_id, Product_name, Product_description, Product_type, Brand) VALUES
(1, 'Smartphone', 'Latest model with 256GB storage', 'Physical', 'BrandX'),
(2, 'E-Book', 'Best-selling novel', 'Digital', 'BrandY');
```

```
INSERT INTO SHOP_PRODUCT (SKU, User_id, Product_id, Price, Stock, Discount, Added_date) VALUES
(1001, 1, 1, 599.99, 10, 0.00, '2023-03-01'),
(1002, 4, 2, 9.99, 50, 0.00, '2023-03-05');
```

```
INSERT INTO `ORDER` (Order_id, User_id, Cart_id, Payment_method_id, Coupon_code, Logistic_id, Warehouse_id, Order_status,
Order_date, Total_amount, Delivery_type, Premium_benefit) VALUES
(1, 2, 1, 1, 'DISCOUNT10', 1, 1, 'Shipped', '2023-03-17 10:00:00', 549.99, 'Standard', TRUE);
```

```
INSERT INTO CART_ITEM (CartItem_id, User_id, Cart_id, SKU, Product_id, Cprice, Quantity) VALUES
(1, 1, 1, 1001, 1, 599.99, 1),
(2, 4, 2, 1002, 2, 9.99, 1);
```

```
INSERT INTO LIST (List_id, User_id, List_name, List_updated_date, Is_shared) VALUES
(1, 2, 'Wish List', '2023-03-20 18:00:00', FALSE),
(2, 3, 'Shopping List', '2024-12-27 17:15:00', TRUE);
```

```
INSERT INTO HAS_ADDR (Address_id, User_id, Address_type) VALUES
(1, 2, 'Shipping Address'),
(2, 3, 'Billing Address');
```

```
INSERT INTO USE_COUPON (Coupon_code, User_id) VALUES
('DISCOUNT10', 2),
('DISCOUNT5', 3);
```

```
INSERT INTO LISTS (Product_id, User_id, List_id) VALUES
(1, 2, 1),
(2, 3, 2);
```

```
INSERT INTO DELIVERED_TO (Logistic_id, Address_id) VALUES
(1, 1),
(2, 2);
```

```
INSERT INTO REVIEWS (Customer_user_id, SKU, Product_id, Seller_user_id, Rating, Comment, Review_date) VALUES
(2, 1002, 2, 4, 3.2, 'Terrible product!', '2023-05-15'),
(3, 1001, 1, 1, 2, 'Terrible!', '2023-05-17'),
(2, 1001, 1, 1, 4.8, 'Great product!', '2023-05-15'),
(3, 1002, 2, 4, 4, 'Good Book!', '2023-05-01');
```

```
INSERT INTO OPERATES_IN (Logistic_id, Warehouse_id) VALUES
(1, 1),
(1, 2),
(2, 3);
```

```
INSERT INTO HAS_POINT (Logistic_id, Pickup_id) VALUES
(2, 1);
```

3) 3 triggers for 3 different tables:

-- Ürün cart'a eklenirse Ürünün stogu azaltilir veya carttan çıkarıldığında arttırılır

DELIMITER \$\$

CREATE TRIGGER decrease_stock_after_insert

AFTER INSERT ON CART_ITEM

FOR EACH ROW

BEGIN

 UPDATE SHOP_PRODUCT

 SET Stock = Stock - NEW.Quantity

 WHERE SKU = NEW.SKU AND Product_id = NEW.Product_id AND User_id = NEW.User_id;

END\$\$

DELIMITER ;

DELIMITER \$\$

CREATE TRIGGER increase_stock_after_delete

AFTER DELETE ON CART_ITEM

FOR EACH ROW

BEGIN

 UPDATE SHOP_PRODUCT

 SET Stock = Stock + OLD.Quantity

 WHERE SKU = OLD.SKU AND Product_id = OLD.Product_id AND User_id = OLD.User_id;

END\$\$

DELIMITER ;

-- Sepetteki toplam ücreti hesaplar

DELIMITER \$\$

CREATE TRIGGER update_cart_total_amount_and_order_total

AFTER INSERT ON CART_ITEM

FOR EACH ROW

BEGIN

-- Step 1: Update Cart_total_amount in the CART table

UPDATE CART

SET Cart_total_amount = (

SELECT SUM(Cprice * Quantity)

FROM CART_ITEM

WHERE Cart_id = NEW.Cart_id

)

WHERE Cart_id = NEW.Cart_id;

-- Step 2: Update Total_amount in the ORDER table based on updated Cart_total_amount

UPDATE `ORDER`

SET Total_amount = (

SELECT Cart_total_amount

FROM CART

WHERE Cart_id = NEW.Cart_id

)

WHERE Cart_id = NEW.Cart_id;

END \$\$

DELIMITER ;

-- Satıcı için yeni ortalama derecelendirmeyi hesaplar ve günceller

DELIMITER \$\$

CREATE TRIGGER update_seller_rating_after_insert

AFTER INSERT ON REVIEWS

FOR EACH ROW

BEGIN

UPDATE SELLER

SET Rating = (

SELECT AVG(Rating)

FROM REVIEWS

```

        WHERE Seller_user_id = NEW.Seller_user_id
    )

    WHERE User_id = NEW.Seller_user_id;

END$$

DELIMITER ;

DELIMITER $$

CREATE TRIGGER update_seller_rating_after_delete
AFTER DELETE ON REVIEWS
FOR EACH ROW
BEGIN
    -- Update the seller's rating after a review is deleted

    UPDATE SELLER

    SET Rating = (

        SELECT IFNULL(AVG(Rating), 0)

        FROM REVIEWS

        WHERE Seller_user_id = OLD.Seller_user_id

    )

    WHERE User_id = OLD.Seller_user_id;

END$$

DELIMITER ;

```

4) 3 meaningful check constraints:

```

CREATE TABLE SHOP_PRODUCT (
    SKU INT,
    User_id INT,
    Product_id INT,
    Price DECIMAL(10, 2),
    Stock INT,
    Discount DECIMAL(10, 2),
    Added_date DATETIME,
    PRIMARY KEY (SKU, User_id, Product_id),
    FOREIGN KEY (User_id) REFERENCES SELLER(User_id),
    FOREIGN KEY (Product_id) REFERENCES PRODUCT(Product_id),
    CHECK(STOCK > -1),
    CHECK(Price > 0)
);

```

CHECK(STOCK > -1) AND CHECK(PRICE > 0)

```
CREATE TABLE CART_ITEM (  
    CartItem_id INT,  
    User_id INT,  
    Cart_id INT,  
    SKU INT,  
    Product_id INT,  
    Cprice DECIMAL (10, 2),  
    Quantity INT,  
    PRIMARY KEY (CartItem_id, User_id, SKU, Cart_id, Product_id),  
    FOREIGN KEY (SKU) REFERENCES SHOP_PRODUCT(SKU),  
    FOREIGN KEY (User_id) REFERENCES SELLER(User_id),  
    FOREIGN KEY (Cart_id) REFERENCES CART(Cart_id),  
    FOREIGN KEY (Product_id) REFERENCES PRODUCT(Product_id),  
    CHECK(Quantity > 0)  
);
```

CHECK(QUANTITY > 0)

```
CREATE TABLE REVIEWS (  
    Customer_user_id INT,  
    SKU INT,  
    Product_id INT,  
    Seller_user_id INT,  
    Rating DECIMAL(3, 2),  
    Comment TEXT,  
    Review_date DATETIME,  
    PRIMARY KEY (Seller_user_id, Customer_user_id, SKU, Product_id),  
    FOREIGN KEY (Customer_user_id) REFERENCES CUSTOMER(User_id),  
    FOREIGN KEY (SKU) REFERENCES SHOP_PRODUCT(SKU),  
    FOREIGN KEY (Product_id) REFERENCES PRODUCT(Product_id),  
    FOREIGN KEY (Seller_user_id) REFERENCES SELLER(User_id),  
    CHECK (Rating >= 0 AND Rating <= 5)  
);
```

CHECK(Rating >= 0 AND Rating <= 5)

5) SQL Statements:

a. *INSERT, DELETE and UPDATE statements for 3 of the tables*

```
DELETE FROM REVIEWS;
```

```
DELETE FROM REVIEWS WHERE Customer_user_id = 2 AND SKU = 1002;
```

```
INSERT INTO REVIEWS (Customer_user_id, SKU, Product_id, Seller_user_id, Rating, Comment, Review_date) VALUES
```

```
(2, 1002, 2, 4, 3.2, 'Terrible Product!', '2023-05-15'),
```

```
(3, 1001, 1, 1, 2, 'Terrible!', '2023-06-17'),
```

```
(2, 1001, 1, 1, 4.8, 'Great product!', '2023-05-15'),
```

```
(3, 1002, 2, 4, 4, 'Good Book!', '2023-05-01');
```

```
SELECT * FROM REVIEWS;
```

```
SELECT * FROM SELLER;
```

```
UPDATE REVIEWS
```

```
SET comment = 'I really liked it'
```

```
WHERE Customer_user_id = 2 AND SKU=1002 ;
```

```
DELETE FROM CART_ITEM WHERE Cart_id = 2;
```

```
INSERT INTO CART_ITEM (CartItem_id, User_id, Cart_id, SKU, Product_id, Cprice, Quantity) VALUES
```

```
(2, 4, 2, 1002, 2, 9.99, 1),
```

```
(3, 4, 2, 1001, 2, 599.99, 1);
```

```
SELECT * FROM CART;
```

```
SELECT * FROM CART_ITEM;
```

```
SELECT * FROM SHOP_PRODUCT;
```

```
UPDATE CART_ITEM
```

```
SET Quantity = 3
```

```
WHERE CartItem_id = 1 AND Cart_id = 1 ;
```

```
SELECT * FROM `ORDER`;
```

```
INSERT INTO PICK_UP_POINT (Pickup_id, PName, Pickup_location)
```

```
VALUES
```

```
(1,'Pickup Point A', '123 Main St, City A'),
```

```
(2,'Pickup Point B', '456 Second St, City B'),
```

```
(3,'Pickup Point C', '789 Third St, City C');
```

```
DELETE FROM OPERATES_IN_PICKUP;
```

```
DELETE FROM PICK_UP_POINT ;
```

```
UPDATE PICK_UP_POINT
```

```
SET Pickup_location = '999 New Street, New City B'
```

```
WHERE Pickup_id = 2;
```

```
SELECT * FROM PICK_UP_POINT;
```

b. 5 SELECT statements for the database (2 of them use 2 tables, 3 of them use 3 tables):

```
-- 2 tables
```

```
SELECT U.User_id, U.Fname, U.Lname, U.Email, CT.Cart_total_amount
```

```
FROM USER U
```

```
JOIN CART CT ON U.User_id = CT.User_id;
```

```
SELECT CI.Cart_id, P.Product_name, CI.Quantity
```

```
FROM CART_ITEM CI
```

```
JOIN PRODUCT P ON CI.Product_id = P.Product_id
```

```
WHERE CI.Cart_id = 1;
```

-- 3 tables

```
SELECT O.Order_id, U.Fname, U.Lname, L.Status, O.Total_amount
FROM `ORDER` O
JOIN USER U ON O.User_id = U.User_id
JOIN LOGISTICS L ON O.Logistic_id = L.Logistic_id;
```

```
SELECT SP.SKU, P.Product_name, P.Product_description, S.User_id, S.Seller_rating, S.Contact_number
FROM SHOP_PRODUCT AS SP
JOIN PRODUCT P ON SP.Product_id = P.Product_id
JOIN SELLER S ON SP.User_id = S.User_id
WHERE S.User_id = 1;
```

```
SELECT U.User_id, U.Fname, U.Lname, CP.Coupon_code, CP.Discount_amount
FROM USER U
JOIN USE_COUPON UC ON U.User_id = UC.User_id
JOIN COUPON CP ON UC.Coupon_code = CP.Coupon_code;
```

c. 5 original SELECT statements

-- 100 tlnin üzerinde alışveriş yapan

```
SELECT
    C.User_id,
    U.Fname,
    U.Lname,
    C.User_id,
    Cart.Cart_id,
    Cart.Cart_total_amount
FROM
    CUSTOMER C
JOIN
    USER U ON C.User_id = U.User_id
JOIN
    CART ON C.User_id = CART.User_id
WHERE
    Cart.Cart_total_amount > 100;
```

-- Satıcının en yüksek puanlı ürünü ve satıcı

```
SELECT
    SP.User_id AS Seller_id,
    U.Fname AS Seller_FirstName,
    SP.Product_id,
    P.Product_name,
    MAX(R.Rating) AS Highest_Rating
FROM
    SHOP_PRODUCT SP
JOIN
    REVIEWS R ON SP.SKU = R.SKU
JOIN
    PRODUCT P ON SP.Product_id = P.Product_id
```

JOIN

USER U ON SP.User_id = U.User_id

GROUP BY

SP.User_id, SP.Product_id

ORDER BY

Highest_Rating DESC;

-- En çok logistic yapan 2 Depo

SELECT

W.Warehouse_id,

W.Location,

COUNT(OL.Logistic_id) AS Total_Logistics_Operations

FROM

WAREHOUSE W

LEFT JOIN

OPERATES_IN OL ON W.Warehouse_id = OL.Warehouse_id

GROUP BY

W.Warehouse_id

ORDER BY

Total_Logistics_Operations DESC

LIMIT 2;

-- premium üyelerin toplam harcaması

SELECT

C.User_id,

U.Fname,

U.Lname,

SUM(O.Total_amount) AS Total_Spending


```
FROM
    CUSTOMER C
JOIN
    PREMIUM P ON C.User_id = P.User_id
JOIN
    USER U ON C.User_id = U.User_id
JOIN
    `ORDER` O ON C.User_id = O.User_id
GROUP BY
    C.User_id, U.Fname, U.Lname
ORDER BY
    Total_Spending DESC;
```

-- Siparişin detayları

```
SELECT
    C.User_id AS CustomerID,
    U.Fname AS FirstName,
    U.Lname AS LastName,
    O.Order_id AS OrderID,
    O.Total_amount AS TotalAmount,
    O.Delivery_type AS DeliveryType,
    O.Order_date AS OrderDate
FROM
    CUSTOMER C
JOIN
    `ORDER` O ON C.User_id = O.User_id
JOIN
    USER U ON C.User_id = U.User_id
ORDER BY
    O.Order_date DESC;
```

Thank You